

Steering Control (Group B)

General Objectives

- ❖ Design a closed-loop DC motor controller and determine necessary coefficient values
- ❖ Simulate open vs. closed-loop system continuous time response using MATLAB
- ❖ Determine system stability by its transfer function using the Nyquist stability criterion
 - Simulate the discretized closed-loop system model using Simulink software; compare with continuous time MATLAB results
- ❖ Implementation of micro-controller based closed-loop DC motor control
- ❖ Testing and validation of the complete control system

Main Specifications

- ❖ Servo voltage: 4.8V - 6V
- ❖ Communication frequency: 50Hz
- ❖ Servo Rotational range: 180°
- ❖ Feedback: LC circuit for a given inductance (at 20 kHz)

Background

We have finally modeled the servo motor to be used with this system using estimation techniques. This should have produced a system transfer function that relates the input control signal to output steering angle. The second part of the system being a dynamic model, given in state space representation. The next part, how do we combine these two to effectively achieve the overall forward path transfer function. First we will need to convert the dynamic model into a transfer function.

$$\frac{d}{dt} \begin{bmatrix} y \\ \dot{y} \\ \varphi \\ \dot{\varphi} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{2C_{\alpha f} - 2C_{\alpha r}}{mV_x} & 0 & -V_x - \frac{2C_{\alpha f}l_f - 2C_{\alpha r}l_r}{mV_x} \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{2C_{\alpha f}l_f - 2C_{\alpha r}l_r}{I_zV_x} & 0 & -\frac{2C_{\alpha f}l_f^2 - 2C_{\alpha r}l_r^2}{I_zV_x} \end{bmatrix} \begin{bmatrix} y \\ \dot{y} \\ \varphi \\ \dot{\varphi} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{2C_{\alpha f}}{m} \\ 0 \\ \frac{2l_f C_{\alpha f}}{I_z} \end{bmatrix} \delta$$

Equation 9 – Dynamics of an Ackerman steering car under the bicycle model assumptions

Given the dynamic model, Equation 9, one can predict the car's location but this is not enough for control design. We need to describe the system in terms of an error. In this case, two errors. The first will be the horizontal distance between the reference point on the track and the center of mass of the vehicle. This will be the first state (e_1) in the error model, with the second being the change of the state ($\dot{e}_1$).

$$\dot{e}_1 = \dot{y} + V_x(\varphi - \varphi_{des}) = \dot{y} + V_x e_2$$

Equation 10 – Definition of the distance error between the car and track

This is very similar to the dynamic model given above, and following this logic, define the second error as the heading difference between the car and the track. Again this will be two states, one to represent the error state (e_2) and one to represent the change in the error state ($\dot{e}_2$).

$$e_2 = \varphi - \varphi_{des}$$

Equation 11 – Definition of the heading error between the car and track

One will note that at this point, if the speed of the vehicle is not constant from Equation 10, the system is non-linear and we cannot design a control system with the tools given in this course. Therefore, it is imperative that the motor group designs a speed control system to keep the car at a known constant speed. One will also find in the reference text, that a large slip angle will cause the system to be non-linear by the bicycle model assumptions. One of which is the small angle assumption. One will also note missing terms in from [1] in the state space model, due to the fact we are designing for a strait road. If one prefers a more sophisticated system, this is available in detail from [1] in chapter 3.

From these error definitions one can come up with the following state space model of the system. For more detail refer to [1] chapters 2 and 3. The dynamic model of the system in terms of error with respect to the road is as follows in Equation 12.

$$\frac{d}{dt} \begin{bmatrix} e_1 \\ \dot{e}_1 \\ e_2 \\ \dot{e}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{2C_{\alpha f} + 2C_{\alpha r}}{mV_x} & \frac{2C_{\alpha f} + 2C_{\alpha r}}{m} & -\frac{2C_{\alpha f}l_f - 2C_{\alpha r}l_r}{mV_x} \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{2C_{\alpha f}l_f - 2C_{\alpha r}l_r}{I_z V_x} & \frac{2C_{\alpha f}l_f - 2C_{\alpha r}l_r}{I_z} & -\frac{2C_{\alpha f}l_f^2 + 2C_{\alpha r}l_r^2}{I_z V_x} \end{bmatrix} \begin{bmatrix} e_1 \\ \dot{e}_1 \\ e_2 \\ \dot{e}_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{2C_{\alpha f}}{m} \\ 0 \\ \frac{2l_f C_{\alpha f}}{I_z} \end{bmatrix} \delta$$

Equation 12 – Dynamic error model of an Ackerman steering car under bicycle model assumptions

From Equation 12, one can develop the necessary transfer function to use linear control theory for designing a system with the state equation. However, in order to produce a transfer function, an observation equation is needed. This can be defined as Equation 13, where the observation is some distance ' d_s ' ahead of the sensor. This equation uses the small angle assumption so that ' e_2 ' can be directly represented without the sin term.

$$y = e_1 + d_s e_2$$

Equation 13 – The observation equation for the Ackerman steering car

Now, the state space model is complete and can be converted to a transfer function. There is a simple command in Matlab to perform this action, however, one should know the underlying theory for a linear state space model. The Laplace transform of a state space system is given below in Equation 14.

$$\begin{aligned} \dot{x}(t) &= Ax(t) + Bu(t) & sX(s) &= AX(s) + BU(s) \\ y(t) &= Cx(t) + Du(t) & \leftrightarrow Y(s) &= CX(s) + DU(s) \end{aligned}$$

Equation 14 – Laplace Transform of generic linear state space system

One can rearrange the state equation to form Equation 15, where ‘I’ is an identity matrix. This matrix allows the s to be distributed to each state independently.

$$(sI - A)X(s) = BU(s)$$

Equation 15 – State equation rearranged to solve for X(s)

After Equation 15, one can solve for ‘X(s)’ and get Equation 16.

$$X(s) = (sI - A)^{-1}BU(s)$$

Equation 16 – State equation solved for X(s)

This can then be inserted into the observation equation to get Equation 17.

$$Y(s) = C(sI - A)^{-1}BU(s) + DU(s)$$

Equation 17 – Observation equation with the state replaced by the state equation.

Finally one solves for the transfer function algebraically to get a linear transfer function described by Equation 18.

$$\frac{Y(s)}{U(s)} = C(sI - A)^{-1}B + D$$

Equation 18 – Transfer function of a linear state space representation.

The overall block diagram of the system can be seen in Figure 5.

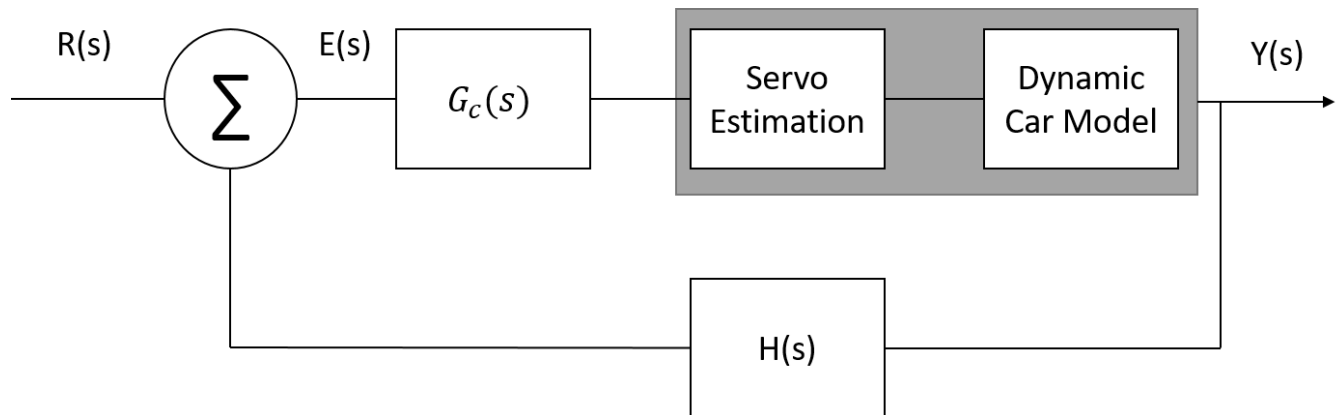


Figure 5 – Overall System Block Diagram for steering

Now with the transfer function found on needs to design a controller. Read the derivation above up to Equation 6. The only difference between what is needed here starts at Equation 6. The first thing, the transfer function for a Proportional Derivative controller is shown below in Equation 19.

$$G_c(s) = k_p + k_d s$$

Equation 19 – Transfer Function of a PD controller

However nothing has changed other than this as we represent the controller generically as 'G_c(s)' and the plant as 'G_p(s)'. Given this information on can take Equation 6 and replace the PI transfer function of 'G_c(s)' with the PD version. This will result in Equation 20, below.

$$G_c(j\omega_c) = \frac{1e^{j(\pi+\varphi_m)}}{|G(j\omega_c)|e^{j\angle G(j\omega_c)}} = \frac{1}{|G(j\omega_c)|} e^{j(\pi+\varphi_m-\angle G(j\omega_c))} = k_p + k_d(j\omega_c)$$

Equation 20 – Algebraic solution for the control design

As done above, the equivalency of the two equation results in Equation 21, below.

$$k_p = \operatorname{Re} \left\{ \frac{1}{|G(j\omega_c)|} e^{j(\pi+\varphi_m-\angle G(j\omega_c))} \right\} = \frac{1}{|G(j\omega_c)|} \cos(\pi + \varphi_m - \angle G(j\omega_c))$$

$$k_d = \frac{1}{j\omega_c} * \operatorname{Im} \left\{ \frac{1}{|G(j\omega_c)|} e^{j(\pi+\varphi_m-\angle G(j\omega_c))} \right\} = \frac{\omega_c^{-1}}{|G(j\omega_c)|} \sin(\pi + \varphi_m - \angle G(j\omega_c))$$

Equation 21 – Design equations for calculation of controller gains

From Equation 21 and the known frequency response of the plant transfer function at the crossover frequency one can calculate the necessary gains for a given phase margin and crossover frequency.

Tasks: Steering Control Design and Simulation

- ❖ Research data sheets for components and understand why they are used (their ratings)
- ❖ Determine appropriate gains (conversion factors) present in your system (i.e. gear ratio, frequency-to-voltage, ADC, etc.) put into Matlab
 - The ADC gain can be noted as a normalized value due to mbed software.
 - Normally you would need the associated bit value representation for the Analog to Digital Converter (ADC) and the Digital to Analog Converter (DAC).
- ❖ **UNDERSTAND** the provided Matlab code and each plotted figure, explanations will be expected.
 - You may need to alter some code in order to get more effective plots.
 - Make note of the resulting 'kp' and 'kd' values in your lab notebook.
 - Determine stability from bode plot
 - Determine stability from Nyquist plot
- ❖ Build system model in Simulink including gains, the feedback loop, and a discretized PD controller.
 - For sample time 'ti' and the controller's unit delay use **1 kHz** for this lab.
- ❖ Plot step response using a three channel scope in Simulink.
 - Step Input
 - Controller Output
 - System Output
- ❖ Inject disturbances to the system and insure rejection from controller.
 - Change in steer angle
 - Introduce disturbance to the angle for one time step
 - Controller Saturation.
 - When the controller exceeds the physical limitation.
 - Be able to explain the concept and show in a plot.

❖ Tabularize results as follows:

➤ Note some information will not be available till the implementation phase.

	Rise time	Overshoot	Undershoot	Steady state error	Settling time
System measurement					
MATLAB (continuous)					
Simulink (discrete)					
Percent Error: (Mat.Vs Sim)					
Percent Error: (Mat. Vs Sys)					
Precent Error (Sim. Vs Sys)					

- ❖ NOTE: All MATLAB and Simulink plots should be included in notebook/reports with discussions
- ❖ Matlab code is provided for the motor team is attached on blackboard.

Tasks: Control Implementation

- ❖ Using mbed, program the microcontroller and test closed-loop controller operation
 - Use First Order Taylor Series approximation method to implement the following PD controller in the discrete domain.
 - Use a software interrupt in order to sample the current position and update the PWM duty cycle at a predetermined interval. Call this interval interrupt time ‘**TI**’.
 - Constants can be set as compiler defines at the top of code or in ‘.h’ files:

```
#define TI 0.001 %1kHz sample time
#define KP 1.0f %Proportional Gain
#define KD 1.0f %Derivative Gain
```

- The PD controller can be represented as follows:

$$u(t) = k_p e(t) + k_D \dot{e}(t)$$

Equation 22 – PI controller in continuous time

- The PD controller with Euler forward approximation can be implemented in code as:

```
errorChange(i)= (error(i) - error(i-1))/TI;
controllerOutput(i) = KP*error(i) + KD*errorChange(i);
```

- Note these are given for a standard PD controller and not a PD + bias controller. You will need to incorporate the bias term.
- ❖ Measure your system’s no-load bandwidth in simulation.

Questions

- 1) What is the fundamental difference between the simulation in Matlab and the simulation in Simulink?
 - a) Hint: Time domain
- 2) Is the controller designed in the discrete time domain or continuous time domain?
 - a) Hint: Have you been taught discrete control design?
- 3) Is the control implementation in the continuous domain? If not, how do we determine the discrete controller is stable in our operational range?
- 4) Is there some way to limit the effect from a sudden in the reference signal of the system?
 - a) Hint: Research Derivative Kick
- 5) How did you determine the no load bandwidth and what frequencies can be rejected?
 - a) Does this match the input waveform performance?

Requirements

- ❖ Check-off requirements:
 - Implantation of controller must use software interrupt (ticker function)
 - Comparison for Simulations and Implementation
 - Tabularized comparison chart completed
 - Description of derivation for control gains
 - Description of stability
 - Bode Plot
 - Nyquist stability criterion
 - Completion and demonstration of control implementation
 - You may push the car with a pole or similar object, trying to maintain a constant set speed.
- ❖ Check-off report: (Group)
 - Refer to rubric and template from Blackboard
 - Be sure to include:
 - Results presentation (include o-scope images, measurement records, etc.)
 - Results analysis and discussion
 - Hardware schematic (using PSpice, TinyCAD, EAGLE, etc.)
 - Problem-solving (describe problems you encountered and how you solved them)

References

- [1] R. Rajamani, Vehicle Dynamics and Control, New York: Springer Science+Business Media, LLC, 2012.